

In-Class Activity 6: Decision Trees, Random Forests, and Boosting

GEOL 599: Machine Learning

Contents

Overview	2
Learning Goals	2
1. Load the Landsat Dataset	2
2. Initial Data Checks	4
Question 1	6
3. Why Decision Trees Have Axis-Aligned Splits	6
4. Build a Two-Feature Dataset for Visualization	6
Question 2	7
5. Fit a Shallow Decision Tree	7
6. Decision Boundary Helper Function	8
Question 3	10
7. Compare Single Tree Depths	10
Question 4	11
8. Compare Tree-Based Ensembles on the Same Two Features	12
Question 5	14
Part 2: Best Practices With All Spectral Features	15
9. Prepare the Full Feature Dataset	15
10. Define Baseline and Candidate Models	15
11. Cross-Validation Comparison	16
Question 6	18
12. Tune XGBoost	18
Question 7	19
13. Final Test-Set Evaluation	19
Question 8	21
14. Feature Importance and Permutation Importance	21
Question 9	23
15. Best-Practice Reflection	24

```
# If needed, install these packages first:
# install.packages(c("tidyverse", "tidymodels", "rpart", "rpart.plot", "ranger", "xgboost", "patchwork")

library(tidyverse)
library(tidymodels)
library(rpart)
library(rpart.plot)
library(ranger)
library(xgboost)
library(patchwork)

tidymodels_prefer()
set.seed(42)
```

Overview

In this activity, you will use the **Statlog Landsat Satellite** dataset to compare several tree-based machine-learning models:

- A single decision tree
- Random forest
- XGBoost

Each row represents a small 3 by 3 pixel neighborhood from Landsat satellite imagery. The predictors are spectral measurements from those pixels, and the target is the land-cover class of the central pixel.

Learning Goals

By the end of this activity, you should be able to:

- Explain why decision trees make **axis-aligned splits**.
- Visualize the limitations of a single decision tree boundary.
- Compare a single tree with ensemble tree methods.
- Explain the difference between bagging and boosting.
- Use train/test splits and cross-validation appropriately.
- Tune tree-based models without leaking information from the test set.
- Interpret model behavior using feature importance and permutation importance.

1. Load the Landsat Dataset

The dataset is the Statlog Landsat Satellite dataset from OpenML/UCI. Each row contains 36 spectral measurements:

```
9 pixels x 4 spectral bands = 36 predictors
```

The four spectral values for the central pixel are attributes 17, 18, 19, and 20. With the naming scheme below, those are:

```
pixel_5_band_1
pixel_5_band_2
pixel_5_band_3
pixel_5_band_4
```

```
data_url <- "https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/Landsat_Landcover.csv"

landsat_raw <- read_csv(data_url, show_col_types = FALSE)

glimpse(landsat_raw)
```

```
## Rows: 6,430
## Columns: 37
## $ pixel_1_band_1 <dbl> 0.117596, -1.205362, 0.779075, 1.146564, -0.764376, -0.~
## $ pixel_1_band_2 <dbl> 1.241362, -1.249654, 0.148811, 0.585831, -1.162250, -0.~
## $ pixel_1_band_3 <dbl> 1.184036, -0.077532, 0.042617, 0.342991, -0.137607, -1.~
## $ pixel_1_band_4 <dbl> 0.815302, 0.444886, -0.243030, 0.021553, 0.180303, -1.0~
## $ pixel_2_band_1 <dbl> -0.158561, -0.895959, 0.800057, 0.947536, -0.969698, -0~
## $ pixel_2_band_2 <dbl> 1.256483, -0.447579, 0.164136, 0.601074, -1.146681, -0.~
## $ pixel_2_band_3 <dbl> 1.193546, -0.786760, 0.053370, 0.353416, -0.126658, -1.~
## $ pixel_2_band_4 <dbl> 0.818486, -0.554203, -0.448612, 0.026550, 0.184937, -1.~
## $ pixel_3_band_1 <dbl> -0.141965, -0.364672, 0.154978, 1.788164, -0.735851, -0~
## $ pixel_3_band_2 <dbl> 0.879481, 0.092157, -0.345245, 1.010702, -1.132569, -0.~
## $ pixel_3_band_3 <dbl> 0.670010, -0.051291, -0.712483, 0.910444, -0.111399, -1~
## $ pixel_3_band_4 <dbl> 0.401020, -0.178503, -0.441923, 0.401020, 0.190284, -1.~
## $ pixel_4_band_1 <dbl> 0.052220, -0.976988, 0.713854, 1.375487, -1.197532, -0.~
## $ pixel_4_band_2 <dbl> 1.204523, -0.852379, 0.504301, 0.810648, -1.202490, -0.~
## $ pixel_4_band_3 <dbl> 1.181239, -0.498640, -0.198662, 0.941256, -0.138666, -1~
## $ pixel_4_band_4 <dbl> 0.758245, 0.124397, -0.403809, 0.230039, 0.388501, -1.1~
## $ pixel_5_band_1 <dbl> -0.151111, -0.668189, 0.735308, 1.104649, -0.668189, -0~
## $ pixel_5_band_2 <dbl> 1.214967, -0.836981, 0.341798, 1.214967, -1.186248, -0.~
## $ pixel_5_band_3 <dbl> 1.187378, -0.487496, -0.188412, 0.409758, -0.367863, -1~
## $ pixel_5_band_4 <dbl> 0.598708, -0.084225, -0.399425, 0.230975, 0.230975, -1.~
## $ pixel_6_band_1 <dbl> -0.136658, -0.656748, 0.160536, 1.126416, -0.879643, 0.~
## $ pixel_6_band_2 <dbl> 1.011130, -0.343506, -0.343506, 1.011130, -0.998975, -0~
## $ pixel_6_band_3 <dbl> 0.899623, -0.176313, -0.355636, 0.720300, -0.355636, -1~
## $ pixel_6_band_4 <dbl> 0.761977, -0.234313, -0.181877, 0.237614, -0.129441, -1~
## $ pixel_7_band_1 <dbl> -0.085593, -0.748307, 0.650756, 1.387105, -0.895577, -0~
## $ pixel_7_band_2 <dbl> 1.211546, -0.981173, -0.016376, 0.860711, -1.244299, -0~
## $ pixel_7_band_3 <dbl> 1.251179, -0.614884, -0.915862, 0.528832, -0.795470, -1~
## $ pixel_7_band_4 <dbl> 0.807707, -0.192752, -0.877277, 0.281150, -0.192752, -0~
## $ pixel_8_band_1 <dbl> -0.069968, -0.736996, 0.671174, 1.412317, -0.885225, -0~
## $ pixel_8_band_2 <dbl> 1.219160, -0.969292, -0.006373, 1.044084, -1.231906, -0~
## $ pixel_8_band_3 <dbl> 1.250463, -0.844805, -0.425752, 0.532085, -0.784941, -1~
## $ pixel_8_band_4 <dbl> 0.597678, -0.400030, -0.662584, 0.282612, -0.347519, -1~
## $ pixel_9_band_1 <dbl> -0.054291, -0.725852, 0.691889, 1.438068, -0.875088, 0.~
## $ pixel_9_band_2 <dbl> 1.233342, -0.344432, 0.356801, 1.058033, -1.220973, -0.~
## $ pixel_9_band_3 <dbl> 1.262255, -0.594534, -0.175259, 0.842981, -0.774223, -1~
## $ pixel_9_band_4 <dbl> 0.603258, -0.183967, -0.236449, 0.130923, -0.551339, -1~
## $ land_cover <dbl> 1, 5, 7, 3, 5, 4, 5, 5, 2, 5, 4, 3, 3, 1, 3, 1, 1, 1, 1~
```

```
feature_cols <- paste0(
  "pixel_",
  rep(1:9, each = 4),
  "_band_",
  rep(1:4, times = 9)
)
```

```
# The target is the last column in this CSV.
```

```
target_col <- names(landsat_raw)[ncol(landsat_raw)]

landsat <- landsat_raw %>%
  rename_with(~ feature_cols, .cols = 1:36) %>%
  rename(land_cover_code = all_of(target_col)) %>%
  mutate(
    land_cover_raw = as.character(land_cover_code),
    land_cover_clean = land_cover_raw %>%
      str_to_lower() %>%
      str_replace_all("[._-]", " ") %>%
      str_squish(),
    land_cover_number = readr::parse_number(land_cover_raw),
    land_cover = case_when(
      land_cover_number == 1 ~ "red_soil",
      land_cover_number == 2 ~ "cotton_crop",
      land_cover_number == 3 ~ "grey_soil",
      land_cover_number == 4 ~ "damp_grey_soil",
      land_cover_number == 5 ~ "soil_with_vegetation_stubble",
      land_cover_number == 7 ~ "very_damp_grey_soil",
      TRUE ~ NA_character_
    ),
    land_cover = factor(land_cover)
  ) %>%
  filter(!is.na(land_cover)) %>%
  select(all_of(feature_cols), land_cover)

landsat %>% slice_head(n = 6)
```

```
## # A tibble: 6 x 37
##   pixel_1_band_1 pixel_1_band_2 pixel_1_band_3 pixel_1_band_4 pixel_2_band_1
##   <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
## 1      0.118         1.24         1.18         0.815        -0.159
## 2     -1.21        -1.25        -0.0775      0.445        -0.896
## 3      0.779         0.149         0.0426     -0.243         0.800
## 4      1.15         0.586         0.343         0.0216        0.948
## 5     -0.764        -1.16        -0.138         0.180        -0.970
## 6     -0.470        -0.725        -1.16        -1.04        -0.232
## # i 32 more variables: pixel_2_band_2 <dbl>, pixel_2_band_3 <dbl>,
## #   pixel_2_band_4 <dbl>, pixel_3_band_1 <dbl>, pixel_3_band_2 <dbl>,
## #   pixel_3_band_3 <dbl>, pixel_3_band_4 <dbl>, pixel_4_band_1 <dbl>,
## #   pixel_4_band_2 <dbl>, pixel_4_band_3 <dbl>, pixel_4_band_4 <dbl>,
## #   pixel_5_band_1 <dbl>, pixel_5_band_2 <dbl>, pixel_5_band_3 <dbl>,
## #   pixel_5_band_4 <dbl>, pixel_6_band_1 <dbl>, pixel_6_band_2 <dbl>,
## #   pixel_6_band_3 <dbl>, pixel_6_band_4 <dbl>, pixel_7_band_1 <dbl>, ...
```

2. Initial Data Checks

Before modeling, check the number of rows, columns, missing values, and class balance.

```
cat("Rows and columns:\n")
```

```
## Rows and columns:
```

```

dim(landsat)

## [1] 6430  37

cat("\nMissing values:\n")

##
## Missing values:

sum(is.na(landsat))

## [1] 0

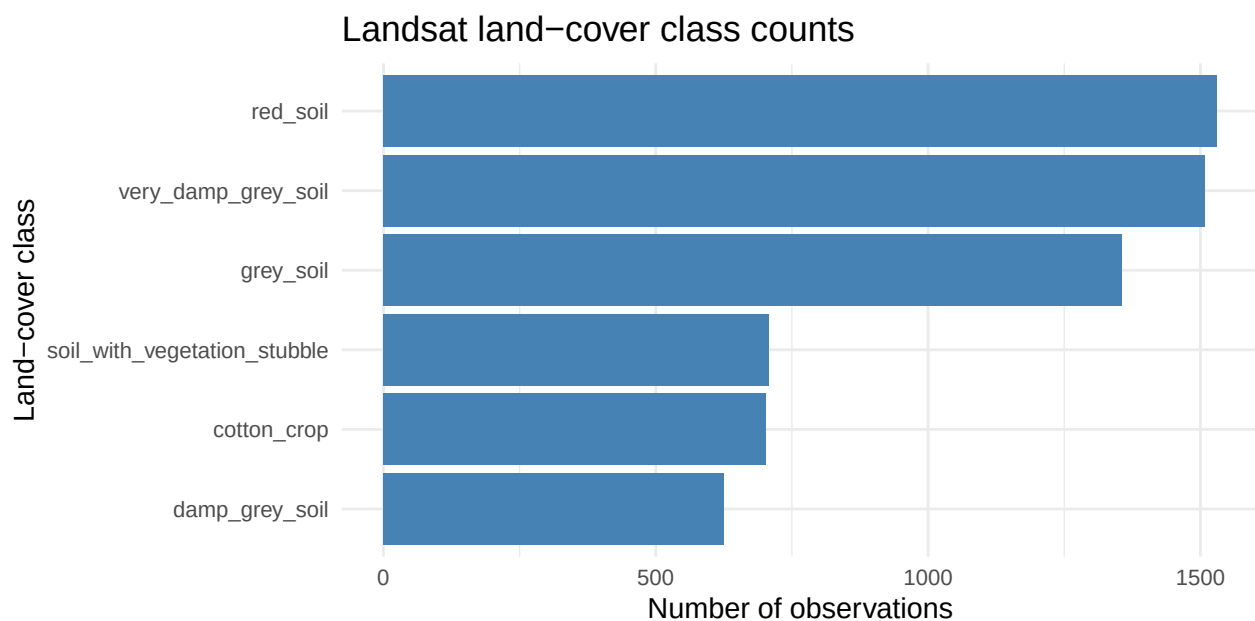
class_counts <- landsat %>%
  count(land_cover, sort = TRUE)

class_counts

## # A tibble: 6 x 2
##   land_cover          n
##   <fct>          <int>
## 1 red_soil        1531
## 2 very_damp_grey_soil 1508
## 3 grey_soil       1356
## 4 soil_with_vegetation_stubble 707
## 5 cotton_crop      703
## 6 damp_grey_soil    625

ggplot(class_counts, aes(x = reorder(land_cover, n), y = n)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Landsat land-cover class counts",
    x = "Land-cover class",
    y = "Number of observations"
  ) +
  theme_minimal(base_size = 13)

```



Question 1

Is this classification problem balanced? Which land-cover classes are most and least common?

This classification problem is not balanced. `damp_grey_soil` is the least common and `red_soil` is the most common, closely followed by `very_damp_grey_soil`. There is a large gap the three most common (`counts>1300`) and the three least common (`counts<710`)

3. Why Decision Trees Have Axis-Aligned Splits

Decision trees split one feature at a time.

Examples:

```
pixel_5_band_1 <= 66
pixel_5_band_2 <= 42
pixel_5_band_4 <= 87
```

That means a decision tree creates boundaries that are vertical or horizontal in a two-feature plot. These are called **axis-aligned splits**.

This is powerful and interpretable, but it can be inefficient when the real class boundary is diagonal, curved, or depends on a smooth combination of predictors.

4. Build a Two-Feature Dataset for Visualization

To visualize decision boundaries, we will use only two predictors from the central pixel:

- `pixel_5_band_1`
- `pixel_5_band_2`

This is intentionally limited. Later, we will use all 36 spectral predictors.

```
two_feature_cols <- c("pixel_5_band_1", "pixel_5_band_2")

landsat_two <- landsat %>%
  select(all_of(two_feature_cols), land_cover) %>%
  drop_na()

set.seed(42)
two_split <- initial_split(landsat_two, prop = 0.75, strata = land_cover)
two_train <- training(two_split)
two_test <- testing(two_split)

class_levels <- levels(landsat_two$land_cover)
class_palette <- setNames(scales::hue_pal()(length(class_levels)), class_levels)

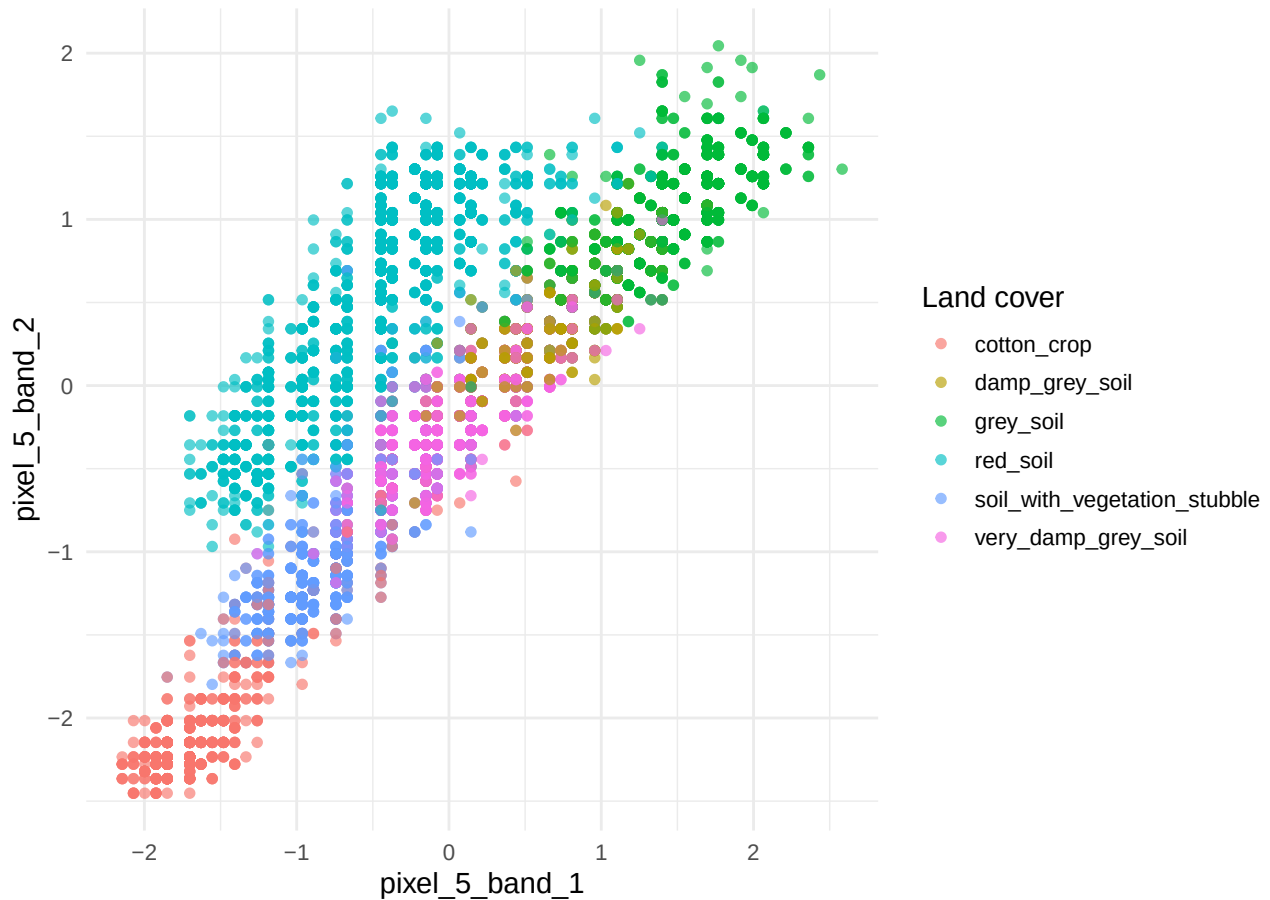
class_palette

##          cotton_crop          damp_grey_soil
##          "#F8766D"          "#B79F00"
##          grey_soil          red_soil
##          "#00BA38"          "#00BFC4"
## soil_with_vegetation_stubble very_damp_grey_soil
##          "#619CFF"          "#F564E3"

ggplot(landsat_two, aes(x = pixel_5_band_1, y = pixel_5_band_2, color = land_cover)) +
  geom_point(alpha = 0.65, size = 1.6) +
```

```
scale_color_manual(values = class_palette) +
labs(
  title = "Landsat classes using two central-pixel spectral bands",
  x = "pixel_5_band_1",
  y = "pixel_5_band_2",
  color = "Land cover"
) +
theme_minimal(base_size = 13)
```

Landsat classes using two central-pixel spectral bands



Question 2

Looking at the scatterplot, where might a single decision tree struggle? Do the classes appear separable by only horizontal and vertical cuts?

The classes cannot be separated by only horizontal and vertical cuts. The red_soil would be easiest to separate as it has a higher value for band 2 while overlapping all other land cover's band 1 values. All other classifications of land cover experience varying levels of overlap between each class. Damp_grey_soil experiences the most overlap of band 1 and 2 values with very_damp_grey_soil and grey_soil.

5. Fit a Shallow Decision Tree

A shallow tree is easy to interpret, but it may be too simple.

```

shallow_tree <- rpart(
  land_cover ~ pixel_5_band_1 + pixel_5_band_2,
  data = two_train,
  method = "class",
  control = rpart.control(maxdepth = 2, cp = 0.001)
)

shallow_tree_pred_train <- predict(shallow_tree, newdata = two_train, type = "class")
shallow_tree_pred_test <- predict(shallow_tree, newdata = two_test, type = "class")

mean(shallow_tree_pred_train == two_train$land_cover)

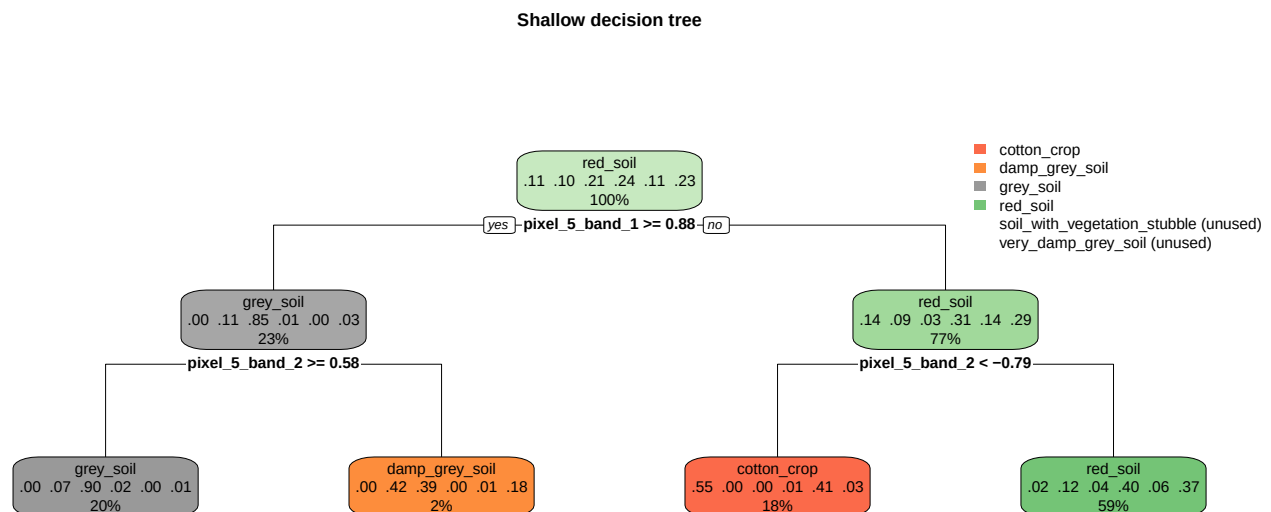
## [1] 0.5263485

mean(shallow_tree_pred_test == two_test$land_cover)

## [1] 0.521118

rpart.plot(
  shallow_tree,
  type = 2,
  extra = 104,
  fallen.leaves = TRUE,
  main = "Shallow decision tree"
)

```



6. Decision Boundary Helper Function

The background color shows the model's predicted class across the two-feature space. The points show the observed classes. Training observations are circles and test observations are X symbols.

```

predict_tree_classes <- function(model, new_data) {
  predict(model, newdata = new_data, type = "class")
}

plot_decision_boundary <- function(model, train_data, test_data, title, predict_fun = predict_tree_classes) {
  x_min <- min(train_data$pixel_5_band_1, test_data$pixel_5_band_1) - 5

```

```

x_max <- max(train_data$pixel_5_band_1, test_data$pixel_5_band_1) + 5
y_min <- min(train_data$pixel_5_band_2, test_data$pixel_5_band_2) - 5
y_max <- max(train_data$pixel_5_band_2, test_data$pixel_5_band_2) + 5

prediction_grid <- expand_grid(
  pixel_5_band_1 = seq(x_min, x_max, length.out = 300),
  pixel_5_band_2 = seq(y_min, y_max, length.out = 300)
)

grid_predictions <- prediction_grid %>%
  mutate(.pred_class = predict_fun(model, prediction_grid))

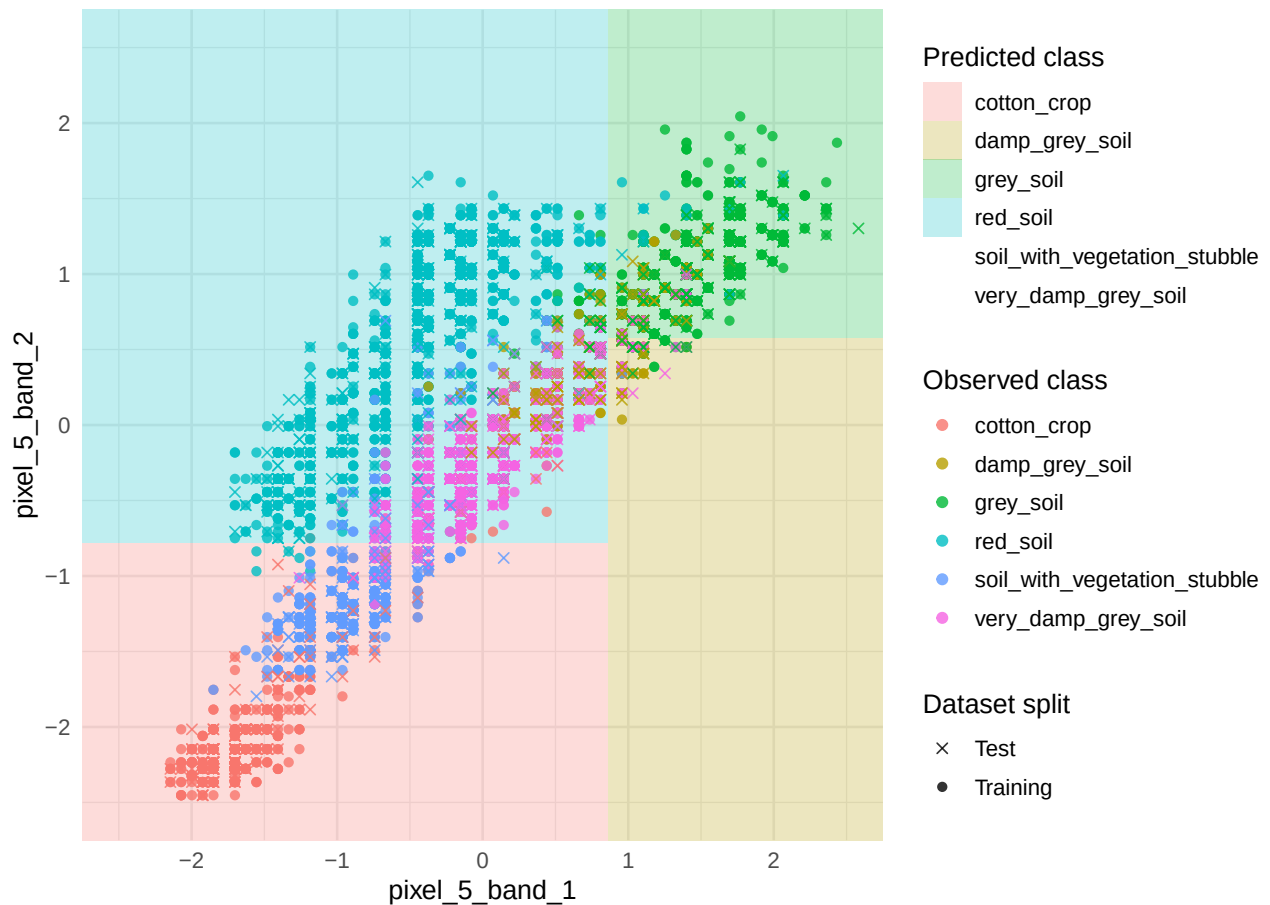
point_data <- bind_rows(
  train_data %>% mutate(split = "Training"),
  test_data %>% mutate(split = "Test")
)

ggplot() +
  geom_raster(
    data = grid_predictions,
    aes(x = pixel_5_band_1, y = pixel_5_band_2, fill = .pred_class),
    alpha = 0.25
  ) +
  geom_point(
    data = point_data,
    aes(x = pixel_5_band_1, y = pixel_5_band_2, color = land_cover, shape = split),
    size = 1.8,
    alpha = 0.8
  ) +
  scale_fill_manual(values = class_palette, drop = FALSE) +
  scale_color_manual(values = class_palette, drop = FALSE) +
  scale_shape_manual(values = c("Training" = 16, "Test" = 4)) +
  coord_cartesian(xlim = c(-2.5, 2.5), ylim = c(-2.5, 2.5)) +
  labs(
    title = title,
    x = "pixel_5_band_1",
    y = "pixel_5_band_2",
    fill = "Predicted class",
    color = "Observed class",
    shape = "Dataset split"
  ) +
  theme_minimal(base_size = 12)
}

plot_decision_boundary(
  shallow_tree,
  two_train,
  two_test,
  "Shallow decision tree: axis-aligned splits"
)

```

Shallow decision tree: axis-aligned splits



Question 3

What do you notice about the decision boundary? Why does it look like rectangles or stair steps?

The decision boundaries look like rectangles and stair steps because they were split only by horizontal or vertically based on the values of band 1 and 2. These boundaries give large prediction areas to categories that have a wider band 1 spread (cotton_crop and red_soil), very poorly predicts damp_grey_soil, and fails to account for 2 other categories.

7. Compare Single Tree Depths

Deeper trees are more flexible, but they can overfit.

```
tree_depths <- c(1, 2, 5, 30)

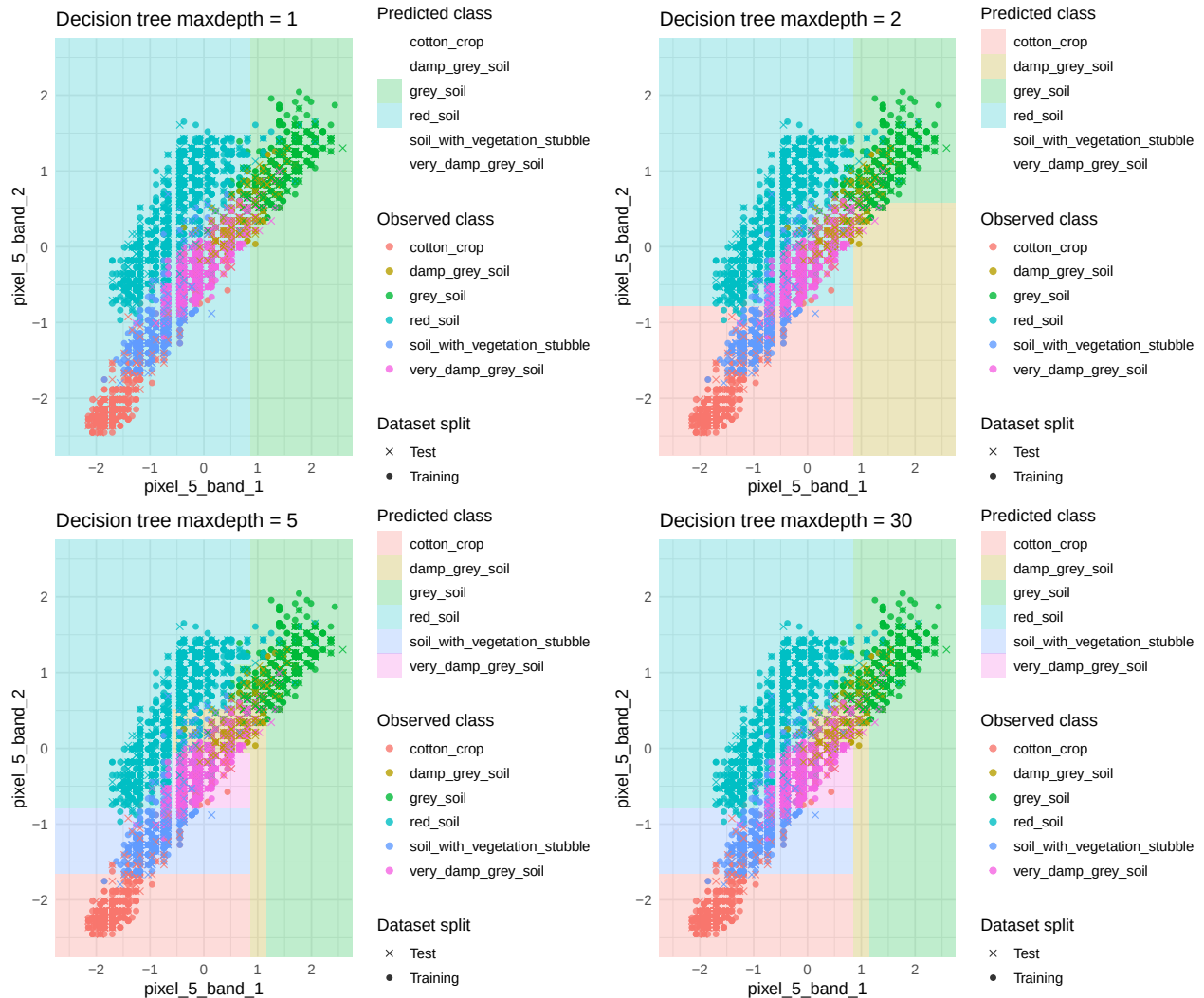
tree_depth_plots <- map(
  tree_depths,
  function(depth) {
    tree_fit <- rpart(
      land_cover ~ pixel_5_band_1 + pixel_5_band_2,
      data = two_train,
      method = "class",
      control = rpart.control(maxdepth = depth, cp = 0.001)
    )
  }
)
```

```

plot_decision_boundary(
    tree_fit,
    two_train,
    two_test,
    paste0("Decision tree maxdepth = ", depth)
)
}
)

wrap_plots(tree_depth_plots, ncol = 2)

```



Question 4

How does the decision boundary change as the tree gets deeper? Which model seems most likely to overfit?

The decision boundaries become more irregular shaped polygons as the tree gets deeper. The model with the max depth of 30 seems more likely to overfit as the polygons for very_damp_grey_soil and damp_grey_soil are narrower to avoid too much overlapping with the red_soil predictions.

8. Compare Tree-Based Ensembles on the Same Two Features

Now compare:

- Decision tree
- Random forest
- XGBoost

This section is still using only two features so the boundaries can be visualized.

```
# Decision tree
boundary_tree <- rpart(
  land_cover ~ pixel_5_band_1 + pixel_5_band_2,
  data = two_train,
  method = "class",
  control = rpart.control(maxdepth = 5, cp = 0.001)
)

# Random forest
boundary_rf <- ranger(
  land_cover ~ pixel_5_band_1 + pixel_5_band_2,
  data = two_train,
  num.trees = 200,
  max.depth = 6,
  probability = FALSE,
  seed = 42
)

# XGBoost through tidymodels
boundary_xgb_spec <- boost_tree(
  trees = 100,
  tree_depth = 2,
  learn_rate = 0.1,
  sample_size = 0.9,
  mtry = 2
) %>%
  set_mode("classification") %>%
  set_engine("xgboost", eval_metric = "mlogloss")

boundary_xgb_wf <- workflow() %>%
  add_formula(land_cover ~ pixel_5_band_1 + pixel_5_band_2) %>%
  add_model(boundary_xgb_spec)

boundary_xgb <- fit(boundary_xgb_wf, data = two_train)

predict_ranger_classes <- function(model, new_data) {
  predict(model, data = new_data)$predictions %>%
    factor(levels = class_levels)
}

predict_workflow_classes <- function(model, new_data) {
  predict(model, new_data, type = "class")$.pred_class %>%
    factor(levels = class_levels)
}
```

```

two_feature_accuracy <- tibble(
  model = c("Decision tree", "Random forest", "XGBoost"),
  train_accuracy = c(
    mean(predict_tree_classes(boundary_tree, two_train) == two_train$land_cover),
    mean(predict_ranger_classes(boundary_rf, two_train) == two_train$land_cover),
    mean(predict_workflow_classes(boundary_xgb, two_train) == two_train$land_cover)
  ),
  test_accuracy = c(
    mean(predict_tree_classes(boundary_tree, two_test) == two_test$land_cover),
    mean(predict_ranger_classes(boundary_rf, two_test) == two_test$land_cover),
    mean(predict_workflow_classes(boundary_xgb, two_test) == two_test$land_cover)
  )
)

```

```
two_feature_accuracy
```

```

## # A tibble: 3 x 3
##   model          train_accuracy test_accuracy
##   <chr>              <dbl>         <dbl>
## 1 Decision tree      0.801           0.786
## 2 Random forest     0.831           0.812
## 3 XGBoost           0.830           0.816

```

```
two_feature_accuracy$train_accuracy-two_feature_accuracy$test_accuracy
```

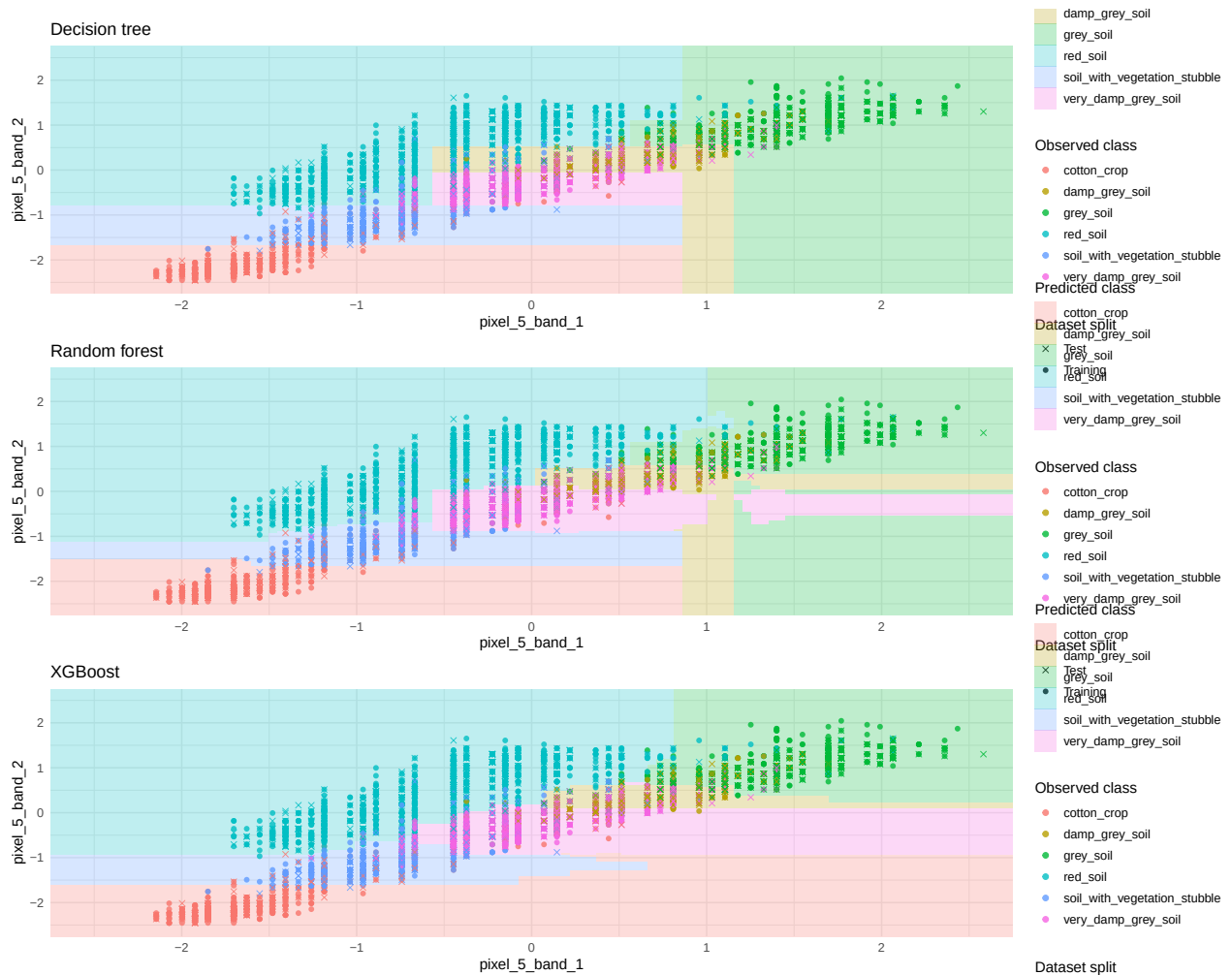
```
## [1] 0.01470194 0.01890544 0.01434886
```

```

p1 <- plot_decision_boundary(boundary_tree, two_train, two_test, "Decision tree")
p2 <- plot_decision_boundary(boundary_rf, two_train, two_test, "Random forest", predict_fun = predict_ranger)
p3 <- plot_decision_boundary(boundary_xgb, two_train, two_test, "XGBoost", predict_fun = predict_workflow)

p1 / p2 / p3

```



Question 5

Compare the three decision boundaries.

1. Which model has the simplest boundary?

The decision tree model has the simplest boundaries as every boundary are mostly rectangular in shape.

2. Which model has the most complex boundary?

The XGBoost model has the most complex boundaries. All vary in shape and thickness throughout each polygon.

3. Which model seems most likely to overfit?

I think the Random forest model is most likely to overfit. The Random Forest model has blocky and disjointed boundaries when compared to the XGBoost model. The Random Forest model also performs better then all the other models on the training data but performs slightly worse than the XGBoost model on the test data.

4. Why do all tree-based boundaries still look somewhat blocky?

The blockiness is from the binary decision-making that the model uses to create the boundaries, all of them are blocky because there are no diagonals.

Part 2: Best Practices With All Spectral Features

The two-feature plots are useful for learning, but they are not the best possible models. In real modeling, we should use a proper train/test split, cross-validation, preprocessing, and tuning.

We will now use all 36 spectral predictors.

Best-practice reminders:

- Keep the test set untouched until final evaluation.
- Use cross-validation for model comparison and tuning.
- Use workflows or consistent preprocessing to avoid leakage.
- Tune complexity controls such as tree depth, number of trees, and learning rate.
- Compare against a simple baseline.

9. Prepare the Full Feature Dataset

```
set.seed(42)
landsat_split <- initial_split(landsat, prop = 0.75, strata = land_cover)
train_data <- training(landsat_split)
test_data <- testing(landsat_split)

nrow(train_data)

## [1] 4820
nrow(test_data)

## [1] 1610
ncol(train_data) - 1

## [1] 36
```

10. Define Baseline and Candidate Models

Tree models do not require feature scaling. We include a median imputer in the recipe as a best-practice safeguard, although this dataset should not have missing predictor values.

```
landsat_recipe <- recipe(land_cover ~ ., data = train_data) %>%
  step_impute_median(all_numeric_predictors())

baseline_spec <- null_model() %>%
  set_mode("classification") %>%
  set_engine("parsnip")

tree_spec <- decision_tree(
  tree_depth = 8,
  min_n = 5,
  cost_complexity = 0.001
) %>%
  set_mode("classification") %>%
  set_engine("rpart")

rf_spec <- rand_forest(
  trees = 300,
```

```

  mtry = 6,
  min_n = 2
) %>%
  set_mode("classification") %>%
  set_engine("ranger")

xgb_spec <- boost_tree(
  trees = 150,
  tree_depth = 2,
  learn_rate = 0.05,
  sample_size = 0.9,
  mtry = 24
) %>%
  set_mode("classification") %>%
  set_engine("xgboost", eval_metric = "mlogloss")

make_workflow <- function(model_spec) {
  workflow() %>%
    add_recipe(landsat_recipe) %>%
    add_model(model_spec)
}

candidate_workflows <- list(
  "Baseline" = make_workflow(baseline_spec),
  "Decision tree" = make_workflow(tree_spec),
  "Random forest" = make_workflow(rf_spec),
  "XGBoost" = make_workflow(xgb_spec)
)

```

11. Cross-Validation Comparison

Use cross-validation on the training set to compare models before touching the test set.

```

set.seed(42)
cv_folds <- vfold_cv(train_data, v = 5, strata = land_cover)

classification_metrics <- metric_set(accuracy, f_meas)

cv_results <- imap_dfr(
  candidate_workflows,
  function(wf, model_name) {
    fit_resamples(
      wf,
      resamples = cv_folds,
      metrics = classification_metrics,
      control = control_resamples(save_pred = TRUE)
    ) %>%
    collect_metrics() %>%
    mutate(model = model_name)
  }
)

```

```

## > A | warning: While computing multiclass `precision()`, some levels had no predicted events
##               (i.e. `true_positive + false_positive = 0`).

```

```

## Precision is undefined in this case, and those levels will be removed from the
## averaged result.
## Note that the following number of true events actually occurred for each
## problematic event level:
## 'cotton_crop': 108, 'damp_grey_soil': 88, 'grey_soil': 203,
## 'soil_with_vegetation_stubble': 108, 'very_damp_grey_soil': 225

## There were issues with some computations A: x1 > B
## (i.e. `true_positive + false_positive = 0`).
## Precision is undefined in this case, and those levels will be removed from the
## averaged result.
## Note that the following number of true events actually occurred for each
## problematic event level:
## 'cotton_crop': 113, 'damp_grey_soil': 95, 'grey_soil': 200,
## 'soil_with_vegetation_stubble': 106, 'very_damp_grey_soil': 221

## There were issues with some computations A: x1 There were issues with some computations A: x1 B
## (i.e. `true_positive + false_positive = 0`).
## Precision is undefined in this case, and those levels will be removed from the
## averaged result.
## Note that the following number of true events actually occurred for each
## problematic event level:
## 'cotton_crop': 102, 'damp_grey_soil': 98, 'grey_soil': 205,
## 'soil_with_vegetation_stubble': 105, 'very_damp_grey_soil': 225

## There were issues with some computations A: x1 B: x1 There were issues with some computations A
## (i.e. `true_positive + false_positive = 0`).
## Precision is undefined in this case, and those levels will be removed from the
## averaged result.
## Note that the following number of true events actually occurred for each
## problematic event level:
## 'cotton_crop': 102, 'damp_grey_soil': 91, 'grey_soil': 208,
## 'soil_with_vegetation_stubble': 110, 'very_damp_grey_soil': 224

## There were issues with some computations A: x1 B: x1 C: x1 There were issues with some computat
## (i.e. `true_positive + false_positive = 0`).
## Precision is undefined in this case, and those levels will be removed from the
## averaged result.
## Note that the following number of true events actually occurred for each
## problematic event level:
## 'cotton_crop': 97, 'damp_grey_soil': 91, 'grey_soil': 207,
## 'soil_with_vegetation_stubble': 99, 'very_damp_grey_soil': 233

## There were issues with some computations A: x1 B: x1 C: x1 D: x1 There were issues with some

```

cv_results

```

## # A tibble: 8 x 7
##   .metric .estimator mean      n std_err .config      model
##   <chr>   <chr>   <dbl> <int>   <dbl> <chr>      <chr>
## 1 accuracy multiclass 0.240     5 0.00129 pre0_mod0_post0 Baseline
## 2 f_meas   macro      0.387     5 0.00167 pre0_mod0_post0 Baseline
## 3 accuracy multiclass 0.856     5 0.00547 pre0_mod0_post0 Decision tree
## 4 f_meas   macro      0.820     5 0.00820 pre0_mod0_post0 Decision tree
## 5 accuracy multiclass 0.910     5 0.00386 pre0_mod0_post0 Random forest
## 6 f_meas   macro      0.890     5 0.00496 pre0_mod0_post0 Random forest
## 7 accuracy multiclass 0.873     5 0.00316 pre0_mod0_post0 XGBoost
## 8 f_meas   macro      0.848     5 0.00422 pre0_mod0_post0 XGBoost

```

```
cv_summary <- cv_results %>%
  select(model, .metric, mean, std_err) %>%
  arrange(.metric, desc(mean))
```

```
cv_summary
```

```
## # A tibble: 8 x 4
##   model      .metric  mean std_err
##   <chr>      <chr>    <dbl>  <dbl>
## 1 Random forest accuracy 0.910 0.00386
## 2 XGBoost    accuracy 0.873 0.00316
## 3 Decision tree accuracy 0.856 0.00547
## 4 Baseline    accuracy 0.240 0.00129
## 5 Random forest f_meas  0.890 0.00496
## 6 XGBoost    f_meas  0.848 0.00422
## 7 Decision tree f_meas  0.820 0.00820
## 8 Baseline    f_meas  0.387 0.00167
```

Question 6

Which model has the best cross-validated macro F1 score? Does any model have much higher training accuracy than validation accuracy? What would that suggest?

The Random forest model has the best cross-validated macro F1 score, but all except for the baseline model would have 'good' F1 scores. No model returned with a higher training accuracy than validation accuracy, if any did, we could consider them overfitted.

12. Tune XGBoost

XGBoost has many tuning parameters. We will tune only a few so the activity stays manageable.

Important parameters:

- `tree_depth`: tree complexity
- `learn_rate`: contribution of each tree
- `trees`: number of boosting rounds
- `sample_size`: fraction of rows used per tree
- `mtry`: number of predictors sampled at each split

Best practice: tune on the training set using cross-validation, then evaluate once on the held-out test set.

```
xgb_tune_spec <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  learn_rate = tune(),
  sample_size = tune(),
  mtry = tune(),
  min_n = 2
) %>%
  set_mode("classification") %>%
  set_engine("xgboost", eval_metric = "mlogloss")

xgb_tune_wf <- workflow() %>%
  add_recipe(landsat_recipe) %>%
  add_model(xgb_tune_spec)
```

```

xgb_grid <- crossing(
  trees = c(150, 300),
  tree_depth = c(2, 4),
  learn_rate = c(0.03, 0.1),
  sample_size = c(0.8, 1.0),
  mtry = c(12, 24)
)

set.seed(42)
xgb_tuned <- tune_grid(
  xgb_tune_wf,
  resamples = cv_folds,
  grid = xgb_grid,
  metrics = classification_metrics,
  control = control_grid(save_pred = TRUE)
)

show_best(xgb_tuned, metric = "f_meas")

## # A tibble: 5 x 11
##   mtry trees tree_depth learn_rate sample_size .metric .estimator mean    n
##   <dbl> <dbl>    <dbl>    <dbl>    <dbl> <chr>    <chr>    <dbl> <int>
## 1    24   300         4        0.1        0.8 f_meas macro    0.897     5
## 2    12   300         4        0.1        0.8 f_meas macro    0.896     5
## 3    12   300         4        0.1         1 f_meas macro    0.893     5
## 4    12   150         4        0.1        0.8 f_meas macro    0.893     5
## 5    24   300         4        0.1         1 f_meas macro    0.891     5
## # i 2 more variables: std_err <dbl>, .config <chr>

best_xgb <- select_best(xgb_tuned, metric = "f_meas")
best_xgb

## # A tibble: 1 x 6
##   mtry trees tree_depth learn_rate sample_size .config
##   <dbl> <dbl>    <dbl>    <dbl>    <dbl> <chr>
## 1    24   300         4        0.1        0.8 pre0_mod31_post0

final_xgb_wf <- finalize_workflow(xgb_tune_wf, best_xgb)

```

Question 7

Which parameters were selected for XGBoost? Were the selected trees shallow or deep? What does sample_size do?

All parameters, except sample size, selected for the final XGBoost were the high end option. This created a lot (300 trees) of relatively deep trees (4). The learn rate was 0.1, mtry was 24, and sample size was 0.8. The sample size is the fraction of rows of data that is used per tree for the model. While our final sample size was lowest suggested for the parameter, this is a higher value for what sample size can equal (usually 0.5-1.0).

13. Final Test-Set Evaluation

Now evaluate the selected models on the test set.

The test set should be used only after model comparison and tuning decisions are complete.

```
final_workflows <- list(
  "Decision tree" = candidate_workflows[["Decision tree"]],
  "Random forest" = candidate_workflows[["Random forest"]],
  "Tuned XGBoost" = final_xgb_wf
)
```

```
final_predictions <- imap_dfr(
  final_workflows,
  function(wf, model_name) {
    fit(wf, data = train_data) %>%
      predict(test_data, type = "class") %>%
      bind_cols(test_data %>% select(land_cover)) %>%
      mutate(model = model_name)
  }
)
```

```
final_predictions %>%
  group_by(model) %>%
  accuracy(truth = land_cover, estimate = .pred_class)
```

```
## # A tibble: 3 x 4
##   model      .metric .estimator .estimate
##   <chr>      <chr>    <chr>      <dbl>
## 1 Decision tree accuracy multiclass    0.861
## 2 Random forest accuracy multiclass    0.920
## 3 Tuned XGBoost accuracy multiclass    0.922
```

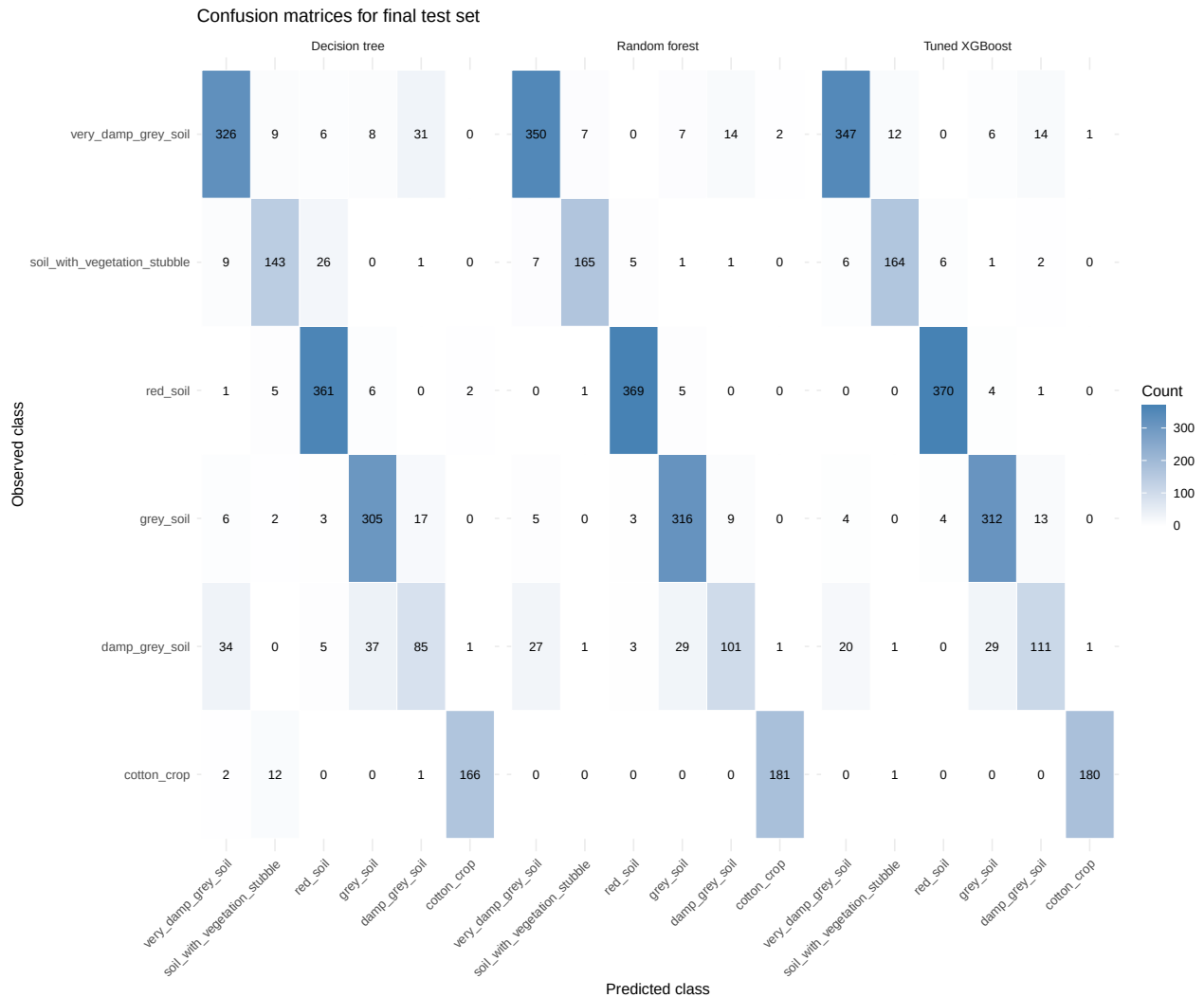
```
final_predictions %>%
  group_by(model) %>%
  f_meas(truth = land_cover, estimate = .pred_class)
```

```
## # A tibble: 3 x 4
##   model      .metric .estimator .estimate
##   <chr>      <chr>    <chr>      <dbl>
## 1 Decision tree f_meas  macro      0.836
## 2 Random forest f_meas  macro      0.905
## 3 Tuned XGBoost f_meas  macro      0.909
```

```
confusion_plot_data <- final_predictions %>%
  count(model, land_cover, .pred_class, name = "n") %>%
  complete(model, land_cover, .pred_class, fill = list(n = 0))
```

```
ggplot(confusion_plot_data, aes(x = .pred_class, y = land_cover, fill = n)) +
  geom_tile(color = "white") +
  geom_text(aes(label = n), size = 3) +
  facet_wrap(~ model) +
  scale_x_discrete(limits = rev) +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(
    title = "Confusion matrices for final test set",
    x = "Predicted class",
    y = "Observed class",
    fill = "Count"
  ) +
  theme_minimal(base_size = 11) +
```

```
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Question 8

Which model performs best on the test set? Is that the same model that performed best during cross-validation? If not, why might that happen?

The tuned XGBoost model performed the best on the test set. The Random forest had performed better during cross-validation. I think the tuning of the XGBoost model added to the increased performance and that the random forest model was at risk of overfitting.

14. Feature Importance and Permutation Importance

Tree-based models often provide built-in feature importance, but those values can be biased. Permutation importance is usually easier to explain:

A feature is important if model performance gets worse when that feature is randomly shuffled.

Here we will use permutation importance on the tuned XGBoost model.

```

final_xgb_fit <- fit(final_xgb_wf, data = train_data)

baseline_predictions <- predict(final_xgb_fit, test_data, type = "class") %>%
  bind_cols(test_data %>% select(land_cover))

baseline_f1 <- f_meas(
  baseline_predictions,
  truth = land_cover,
  estimate = .pred_class
) %>%
  pull(.estimate)

set.seed(42)
permute_one_feature <- function(feature_name, repeats = 10) {
  map_dfr(seq_len(repeats), function(rep_id) {
    permuted_test <- test_data
    permuted_test[[feature_name]] <- sample(permuted_test[[feature_name]])

    permuted_predictions <- predict(final_xgb_fit, permuted_test, type = "class") %>%
      bind_cols(test_data %>% select(land_cover))

    permuted_f1 <- f_meas(
      permuted_predictions,
      truth = land_cover,
      estimate = .pred_class
    ) %>%
      pull(.estimate)

    tibble(
      feature = feature_name,
      repeat_id = rep_id,
      importance = baseline_f1 - permuted_f1
    )
  })
}

importance_df <- map_dfr(feature_cols, permute_one_feature) %>%
  group_by(feature) %>%
  summarize(
    importance_mean = mean(importance),
    importance_sd = sd(importance),
    .groups = "drop"
  ) %>%
  arrange(desc(importance_mean))

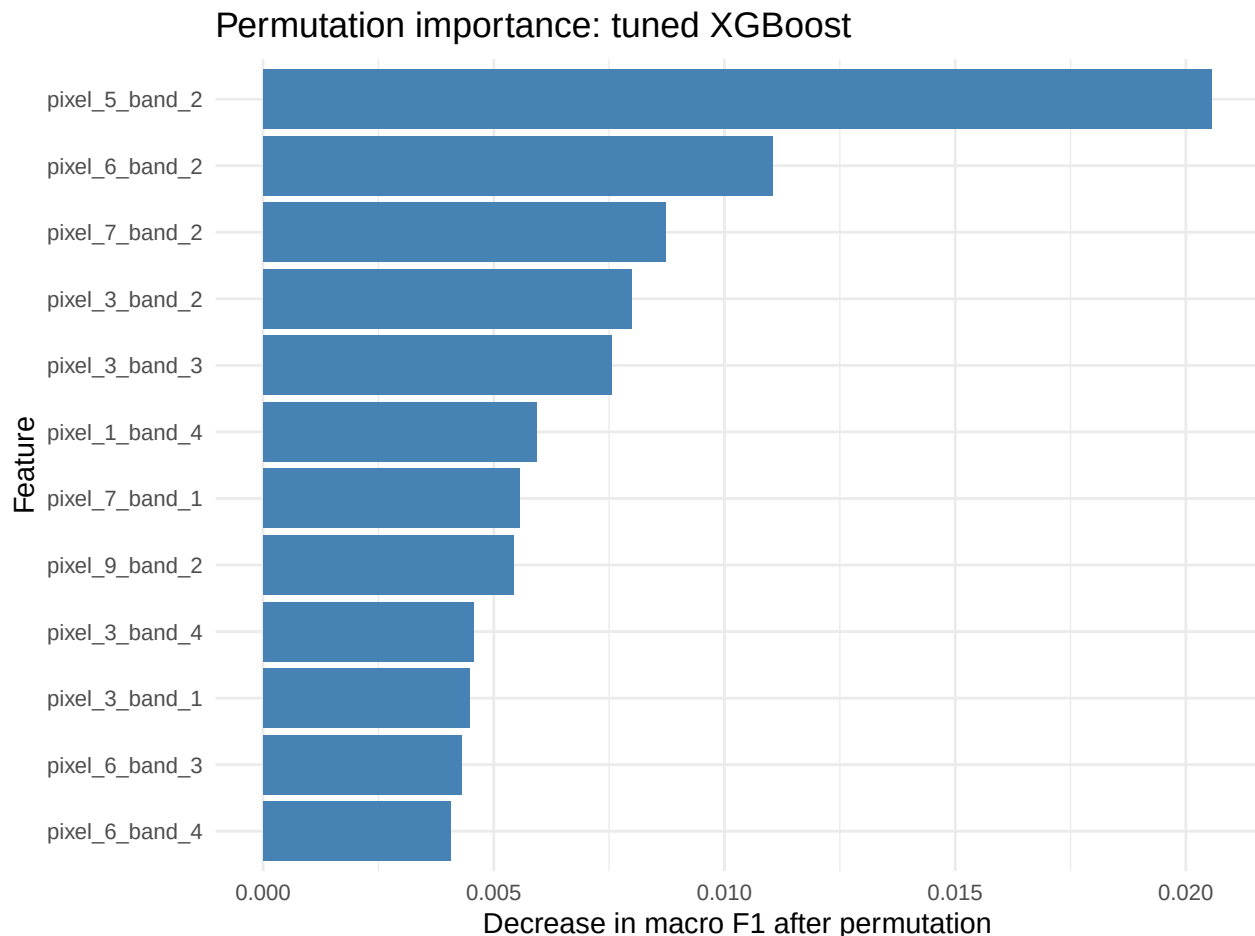
importance_df %>% slice_head(n = 12)

## # A tibble: 12 x 3
##   feature      importance_mean importance_sd
##   <chr>          <dbl>          <dbl>
## 1 pixel_5_band_2      0.0206      0.00533
## 2 pixel_6_band_2      0.0110      0.00248
## 3 pixel_7_band_2      0.00873     0.00460
## 4 pixel_3_band_2      0.00800     0.00295

```

```
## 5 pixel_3_band_3      0.00754      0.00303
## 6 pixel_1_band_4      0.00593      0.00287
## 7 pixel_7_band_1      0.00556      0.00232
## 8 pixel_9_band_2      0.00544      0.00307
## 9 pixel_3_band_4      0.00456      0.00362
## 10 pixel_3_band_1     0.00449      0.00314
## 11 pixel_6_band_3     0.00430      0.00277
## 12 pixel_6_band_4     0.00406      0.00220
```

```
importance_df %>%
  slice_max(importance_mean, n = 12) %>%
  ggplot(aes(x = reorder(feature, importance_mean), y = importance_mean)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Permutation importance: tuned XGBoost",
    x = "Feature",
    y = "Decrease in macro F1 after permutation"
  ) +
  theme_minimal(base_size = 13)
```



Question 9

Which spectral features appear most important? Do the central-pixel bands appear useful? Why might neighboring-pixel spectral values also help classify land cover?

Be careful: feature importance describes what the model used. It does not prove causation.

Pixel 5 band 2 appears to be the most important, but band 2 highly represented in importance of variables. Central-pixel bands are more useful as pixel 5, 6, and 7 are the top three in importance, but pixel 4 is strangely absent. The neighboring-pixel spectral values help classify land cover as there is likely some auto correlation but could also be used to help scale the spectral values to reflect the true color of the targeted pixel that affected by environmental factors (clouds affecting spectral values).

15. Best-Practice Reflection

Answer these questions in a few sentences each.

1. Why is a single decision tree easy to interpret but prone to overfitting?

The single tree is weighted very heavily in a model and has no probability to adjust the decision boundaries.

2. Why do decision tree boundaries look blocky in two dimensions?

The binary decisions, made in the tree, splits the boundaries by horizontal and vertical lines only.

3. How is a random forest different from a single tree?

Random forest does multiple trees and bases the final decision on the majority of results from the forest.

4. How is boosting different from random forest bagging?

Bagging runs in parallel to narrow variance, while boosting corrects bias through tuning.

5. Why should XGBoost be tuned with cross-validation instead of directly on the test set?

Test set is only for the final run to prevent data leakage.

6. Why is permutation importance useful, and what are its limitations?

We can see what the model is relying on. The limitations are that the parameters that it relies on may be from an incorrect assumption and make the model fundamentally wrong.

7. For environmental remote-sensing datasets, why might random train/test splits still be too optimistic?

Spatial and temporal auto correlations and the processed data (depending on what level you use) will be a source of data leakage.

Key Takeaways

- A decision tree makes one-feature-at-a-time splits, creating axis-aligned boundaries.
- A shallow tree may underfit; a deep tree may overfit.
- Random forests average many trees to reduce variance.
- XGBoost builds trees sequentially so later trees correct earlier mistakes.
- XGBoost is powerful, but tuning and validation matter.
- Workflows help prevent preprocessing leakage.
- Feature importance helps explain model behavior, but importance is not causality.
- In environmental science, spatial and temporal validation may matter more than small differences among algorithms.